

SCP-PROB: Probability-Driven Nonlinear Congestion Control for Lossy Paths

Skai Uijing
Independent Researcher
China
skaiuijing@gmail.com

Abstract

Reliable transports commonly treat packet loss and queueing delay as separate congestion signals, even though both increase delivery completion time. This paper introduces SCP-PROB, a new window-based congestion-control algorithm for lossy paths. SCP-PROB converts RTT inflation and expected loss-recovery cost into a unified completion-time residual, maps that residual to a bounded congestion likelihood, and drives a hysteretic nonlinear window controller with the resulting signal. We implement the algorithm in SCP, a reliable user-space transport. On a 5 Mbit/s path with approximately 100 ms unloaded RTT, SCP-PROB achieved 4.698 Mbit/s without random loss, comparable to SCP-AIMD and TCP CUBIC. Under independent 10% loss in both directions, it achieved 0.876 Mbit/s, $2.05\times$ SCP-AIMD and $3.53\times$ CUBIC. Two competing SCP-PROB flows obtained a Jain index of 0.9997. Cross-controller coexistence, however, remained strongly path-dependent on a loss-free deeply buffered path.

CCS Concepts

• **Networks** → **Network protocols**; **Transport protocols**.

Keywords

congestion control, lossy networks, completion time, nonlinear control, reliable transport

1 Introduction

Loss-based congestion controllers reduce their sending window after packet loss, while delay-based controllers react to RTT inflation. Loss is a useful signal when queues overflow, but it is ambiguous on wireless, long-distance, or otherwise unreliable paths: random corruption and transient link failures can trigger the same response as persistent congestion. Delay is available before queue overflow, but a controller that observes only RTT can ignore the substantial recovery time paid by a reliable transport after loss.

The starting point of this work is not that loss and delay are physically identical. A packet that remains in the network may wait in a queue, whereas a lost packet must be retransmitted and recovered. Both mechanisms, nevertheless, increase the time required to complete reliable delivery. We therefore express them in a common completion-time domain.

This paper proposes SCP-PROB, a new congestion-control algorithm with three coupled components. First, it forms an effective residual by adding an estimated loss-recovery cost to the RTT residual. Second, it maps the effective residual through a logistic function to obtain a bounded congestion likelihood. Third, it uses that likelihood in a two-phase nonlinear window controller. A Schmitt-trigger-style hysteresis switches between increase and

decrease phases, while the update magnitude depends on both the likelihood and the current congestion window.

We implement SCP-PROB in SCP, a reliable user-space protocol with cumulative ACKs, selective acknowledgments, retransmission timers, flow control, and timestamp-based RTT measurement. The comparison with SCP-AIMD holds the transport and recovery implementation constant, isolating the congestion-control algorithm. TCP CUBIC provides an external baseline.

The paper makes the following contributions:

- We introduce SCP-PROB, a new probability-driven nonlinear congestion-control algorithm for reliable transport over lossy paths.
- We derive a Bayesian-motivated affine loss-and-delay score and project it into a unified completion-time residual. The resulting bounded likelihood provides a continuous control signal rather than a binary loss event.
- We design a hysteretic two-phase window update whose increase and decrease magnitudes depend on congestion likelihood and window size. We establish boundedness, branch-wise monotonicity, and a same-signal window-gap contraction property for its continuous approximation.
- We evaluate the implementation under clean and high-loss paths. At 10% independent loss in both directions, SCP-PROB improved goodput by $2.05\times$ over SCP-AIMD and $3.53\times$ over CUBIC, while two homogeneous flows achieved a Jain index of 0.9997.

Figure 1 summarizes the feedback loop. RTT measurements and the smoothed loss signal are projected into a common completion-time residual; the resulting likelihood then controls the window.

2 Unified Completion-Time Congestion Likelihood

2.1 Bayesian-Motivated Score

Let H_0 denote a non-congested path state and H_1 a congested state. For sender observations y and priors $\pi_k = P(H_k)$, Bayes' rule gives

$$\log \frac{P(H_1 | y)}{P(H_0 | y)} = \log \frac{\pi_1}{\pi_0} + \log \frac{p(y | H_1)}{p(y | H_0)}. \quad (1)$$

The logistic inverse maps a log-odds score z to

$$q = \sigma(z) = \frac{1}{1 + \exp(-z)}. \quad (2)$$

The implementation is engineering-calibrated rather than fitted from a complete generative model. We therefore use the conservative term *congestion likelihood*; q is not claimed to be a calibrated posterior probability.

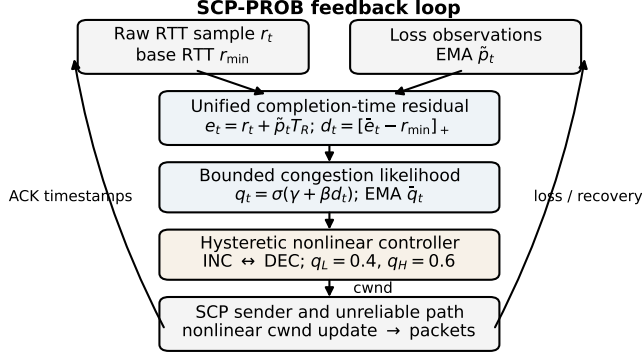


Figure 1: Overview of SCP-PROB. Timestamp-based RTT and smoothed transport-observed loss are projected into a completion-time residual. A bounded likelihood drives the hysteretic nonlinear window controller, and the resulting traffic closes the feedback loop.

Consider a window in which N packets are exposed to the path and K losses are observed, with $\hat{p} = K/N$. Under binomial working models

$$K | H_0 \sim \text{Binomial}(N, a), \quad K | H_1 \sim \text{Binomial}(N, p_C), \quad (3)$$

where $p_C > a$, the loss log-likelihood ratio is exactly affine in $\hat{p}$:

$$L_{\text{loss}}(\hat{p}) = \gamma_L + \alpha \hat{p}, \quad \alpha = N \log \frac{p_C(1-a)}{a(1-p_C)} > 0. \quad (4)$$

This model motivates the affine form; the implementation uses a smoothed transport-observed loss signal. That signal includes recovery pressure seen by SCP and is not intended as an unbiased estimator of the physical link error probability.

Let $d^{(r)} \geq 0$ be the RTT residual before adding recovery cost. A local exponential-tail approximation gives another affine log-likelihood term:

$$L_{\text{delay}}(d^{(r)}) = \gamma_D + \beta d^{(r)}, \quad \beta > 0. \quad (5)$$

The approximation does not assert that Internet RTTs are universally exponential; it supplies a locally monotone model around the selected operating region.

Absorbing constant terms into γ yields

$$z = \gamma + \alpha \hat{p} + \beta d^{(r)}. \quad (6)$$

Since $\beta > 0$, define the equivalent recovery-time cost

$$T_R := \frac{\alpha}{\beta} \quad (7)$$

and the effective residual

$$d^{(e)} = d^{(r)} + \hat{p} T_R. \quad (8)$$

The final control signal is

$$q = \sigma(\gamma + \beta d^{(e)}). \quad (9)$$

Equation (8) is a projection into a common time unit, not a claim of physical equivalence between loss and queueing.

2.2 Online Construction and Calibration

Let r_t be a raw timestamp-based RTT sample, $\tilde{p}_t$ the latest loss EMA, and $r_{\min}$ the minimum observed raw RTT. SCP constructs

$$e_t = r_t + \tilde{p}_t T_R, \quad (10)$$

$$\tilde{e}_t = (1 - \rho_e) \tilde{e}_{t-1} + \rho_e e_t, \quad (11)$$

$$d_t = [\tilde{e}_t - r_{\min}]_+, \quad [x]_+ = \max(0, x). \quad (12)$$

The raw minimum is deliberately kept separate from the effective RTT so that the loss cost is not absorbed into the propagation baseline.

A convenient engineering calibration maps a background effective residual d_0 to q_0 and a residual L milliseconds higher to q_1 :

$$\beta = \frac{\text{logit}(q_1) - \text{logit}(q_0)}{L}, \quad (13)$$

$$\gamma = \text{logit}(q_0) - \beta d_0. \quad (14)$$

The evaluated build used $q_0 = 0.2$, $q_1 = 0.8$, and $L = 100$ ms, with Q16 parameters

$$\gamma_{Q16} = -134460, \quad \beta_{Q16} = 1817. \quad (15)$$

These values map $q = 0.4$ and $q = 0.6$ to effective residuals of approximately 59.4 ms and 88.6 ms, respectively.

The loss signal can have a stable long-run mean while still varying with the window over shorter intervals. A larger window changes packet exposure, queue pressure, ACK behavior, and the amount of data involved in recovery. This coupling is useful: it allows the loss-recovery term to participate in the closed-loop evolution of q and the congestion window.

3 The SCP-PROB Controller

3.1 Hysteretic Phase Selection

Let $s_t \in \{\text{INC}, \text{DEC}\}$ be the controller phase, q_t the instantaneous likelihood, and $\tilde{q}_t$ its EMA. The controller uses thresholds

$$q_L = 0.4, \quad q_H = 0.6. \quad (16)$$

The phase transition is

$$s_{t+1} = \begin{cases} \text{DEC}, & s_t = \text{INC} \text{ and } q_t \geq q_H, \\ \text{INC}, & s_t = \text{DEC} \text{ and } q_t \leq q_L, \\ s_t, & \text{otherwise.} \end{cases} \quad (17)$$

Thus, values in (q_L, q_H) retain phase memory. This hysteresis avoids rapid toggling when the likelihood fluctuates near one threshold.

3.2 Nonlinear Window Update

Let w_t be the congestion window, M the maximum segment size, and b_t the number of newly acknowledged bytes. Define the virtual increase and decrease reference points

$$q_I = \frac{9}{13} \approx 0.6923, \quad q_D = \frac{4}{13} \approx 0.3077, \quad (18)$$

and $k_D = 0.3$. For $[x]_{[0,1]} = \min(1, \max(0, x))$, the increase amount is

$$A(w, q, b) = \max \left\{ 1, \left[1 - \left(\frac{q}{q_I} \right)^2 \right]_{[0,1]} \frac{Mb}{\max(w, 1)} \right\}. \quad (19)$$

The decrease branch uses the larger of instantaneous and smoothed evidence,

$$q^{(d)} = \max(q, \bar{q}), \quad (20)$$

with

$$D(w, q, \bar{q}, b) = k_D \left[\frac{q^{(d)} - q_D}{1 - q_D} \right]_{[0,1]}^2 (w - M) \frac{b}{\max(w, 1)}. \quad (21)$$

After evaluating Equation (17), the window update is

$$w_{t+1} = \begin{cases} \text{clip}_{[M, W_{\max}]}(w_t + A(w_t, q_t, b_t)), & s_{t+1} = \text{INC}, \\ \text{clip}_{[M, W_{\max}]}(w_t - D(w_t, q_t, \bar{q}_t, b_t)), & s_{t+1} = \text{DEC}. \end{cases} \quad (22)$$

The increase branch preserves the familiar property that small windows grow more quickly, but attenuates growth as congestion likelihood rises. The decrease branch acts on the excess window $w - M$, so it cannot drive the window below one segment, and its strength rises quadratically with the congestion signal.

3.3 Analytical Properties

PROPOSITION 1 (BOUNDEDNESS AND MONOTONE RESPONSE). *The implemented window satisfies $M \leq w_t \leq W_{\max}$. Ignoring clipping and the one-byte increase floor, the increase amount is non-increasing in w and q , while the decrease amount is non-decreasing in w , q , and $\bar{q}$.*

PROOF. Boundedness follows from Equation (22). In the increase branch, the unfloored term is proportional to $a(q)/w$, where $a(q) = [1 - (q/q_I)^2]_{[0,1]}$, and therefore decreases with both w and q . In the decrease branch,

$$(w - M) \frac{b}{w} = b \left(1 - \frac{M}{w} \right)$$

is non-decreasing in w , while the squared likelihood factor is non-decreasing in both q and $\bar{q}$. $\square$

PROPOSITION 2 (SAME-SIGNAL BRANCH CONTRACTION). *Consider two flows in the same phase that observe the same frozen q , $\bar{q}$, and newly acknowledged amount b , with $0 < b \leq w$ and $w \geq M$. Ignoring clipping and the increase floor, each active branch is locally non-expansive in the window and strictly contractive whenever its control coefficient is nonzero.*

PROOF. For the increase branch, let $c = a(q)Mb$. Its map is

$$F_I(w) = w + \frac{c}{w}, \quad F'_I(w) = 1 - \frac{c}{w^2}.$$

Since $0 \leq a(q) \leq 1$, $b \leq w$, and $M \leq w$, we have $0 \leq c/w^2 \leq 1$, hence $0 \leq F'_I(w) \leq 1$.

For the decrease branch, let

$$\delta = k_D \left[\frac{q^{(d)} - q_D}{1 - q_D} \right]_{[0,1]}^2 b.$$

Then

$$F_D(w) = w - \delta \left(1 - \frac{M}{w} \right), \quad F'_D(w) = 1 - \frac{\delta M}{w^2}.$$

Because $0 \leq \delta \leq 0.3b \leq 0.3w$ and $M \leq w$, $0.7 \leq F'_D(w) \leq 1$. The inequalities are strict when the corresponding control coefficient is nonzero. $\square$

Algorithm 1 SCP-PROB update on a new ACK

Require: raw RTT r_t , loss EMA $\tilde{p}_t$, base RTT $r_{\min}$, window w_t , phase s_t , newly ACKed bytes b_t

```

1:  $e_t \leftarrow r_t + \tilde{p}_t T_R$ 
2: update smoothed effective RTT  $\bar{e}_t$ 
3:  $d_t \leftarrow \max(0, \bar{e}_t - r_{\min})$ 
4:  $q_t \leftarrow \sigma(\gamma + \beta d_t)$ 
5:  $\bar{q}_t \leftarrow (7\bar{q}_{t-1} + q_t)/8$ 
6: update  $s_{t+1}$  using Equation (17)
7: if  $s_{t+1} = \text{INC}$  then
8:    $w_{t+1} \leftarrow \text{clip}(w_t + A(w_t, q_t, b_t))$ 
9: else
10:   $w_{t+1} \leftarrow \text{clip}(w_t - D(w_t, q_t, \bar{q}_t, b_t))$ 
11: end if
```

The proposition establishes a same-signal fairness tendency, not global fairness across different RTTs, private path components, or controller families. The complete closed loop additionally contains RTT smoothing, loss estimation, recovery dynamics, and phase hysteresis. We therefore do not claim a proof of global convergence or a periodic orbit. In measured high-loss traces, q and w exhibited coupled variation across the controller's operating region.

4 Implementation

SCP is a reliable byte-stream protocol implemented over an unreliable datagram transport. It provides cumulative acknowledgments, a selective acknowledgment gap, retransmission timers, receive-window control, and timestamp echo for RTT sampling. The evaluated maximum segment size was 1430 bytes, and the minimum retransmission timeout was 200 ms.

The SCP-PROB implementation maintains a transport-observed loss estimate and its EMA. Loss does not directly invoke a multiplicative window reduction. Instead, the estimate contributes $\tilde{p}_t T_R$ to the effective RTT sample, changing q and consequently the nonlinear window update. This separation is important on paths where losses are frequent but not necessarily caused by a persistent full queue.

The likelihood EMA uses the update

$$\bar{q}_t = \frac{7\bar{q}_{t-1} + q_t}{8}. \quad (23)$$

All logistic and calibration parameters are stored in Q16 fixed-point form, while the current prototype uses floating-point arithmetic in the nonlinear window update. The algorithm is invoked only for newly acknowledged data.

Algorithm 1 summarizes the ACK fast path.

5 Evaluation

5.1 Methodology

Experiments used three Linux network namespaces on one virtual machine: a client (scp-c), a router (scp-r), and a server (scp-s). The router applied a root netem discipline on each egress direction. Unless otherwise stated, each direction used a 5 Mbit/s rate, 50 ms delay, and a 1000-packet queue. The unloaded RTT was therefore approximately 100 ms. Loss was either disabled or independently

Table 1: Single-flow 100 MiB transfer over a 5 Mbit/s path.

Loss	Controller	Time (s)	Goodput (Mbit/s)	Util. (%)
0%	SCP-PROB	178.576	4.6975	93.95
0%	SCP-AIMD	177.038	4.7383	94.77
0%	TCP CUBIC	175.710	4.7741	95.48
10%	SCP-PROB	958.079	0.8756	17.51
10%	SCP-AIMD	1968.647	0.4261	8.52
10%	TCP CUBIC	3377.570	0.2484	4.97

set to 10% in both directions, affecting data packets and reverse ACKs.

Single-flow experiments transferred 100 MiB (104,857,600 bytes) and report application goodput. SCP completion time was measured at the receiver. The clean-path CUBIC value was measured at the sender and includes waiting for receiver close; differences below two percent in that row should therefore not be interpreted as significant. Fairness experiments used two continuously backlogged flows for 300 seconds and counted bytes delivered to the receiving application. For n flows with goodputs x_i , we use Jain’s index [10]:

$$J = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}. \quad (24)$$

We compare SCP-PROB with SCP-AIMD, which changes only SCP’s congestion controller, and with Linux TCP CUBIC. We also report a NoLossCost ablation that removes $\hat{p}_l T_R$ while retaining the nonlinear window functions.

5.2 Single-Flow File Transfer

Table 1 shows that all three controllers delivered approximately 4.70–4.77 Mbit/s without random loss. Under independent 10% loss in both directions, SCP-PROB achieved 0.8756 Mbit/s, or $2.05\times$ SCP-AIMD and $3.53\times$ CUBIC. Completion time fell by 51.3% and 71.6%, respectively. Because the SCP comparison changes only the congestion-control module, this difference is not caused by a different reliability protocol.

5.3 Measured Control Dynamics

Figures 2–4 show the complete 958-second SCP-PROB run under 10% bidirectional loss. The trace contains 4778 samples, 102 phase transitions, and 50 complete INC–DEC cycles. The mean period was 18.97 s (standard deviation 8.24 s); q ranged from 0.114 to 0.813 and the congestion window from 2.76 to 63.23 KiB.

Figure 2 shows sustained, bounded window regulation rather than application-limited idleness. The mean congestion window was 27.38 KiB and the mean in-flight volume was 28.07 KiB.

The likelihood remained centered near the control region: mean q was 0.4516, while complete cycles had mean minima and maxima of 0.3877 and 0.6368. Thus, the observed signal repeatedly traversed both configured thresholds rather than remaining pinned to one branch.

The overlapping INC and DEC samples in Figure 4 are the measured signature of the Schmitt-trigger design. They should not be interpreted as a proof of a deterministic limit cycle; the networked

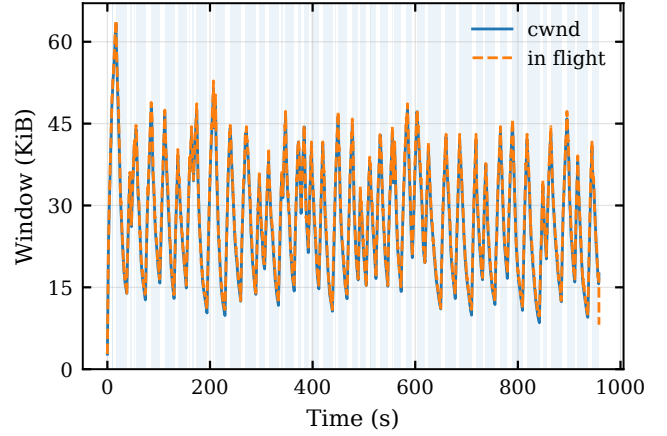


Figure 2: Congestion window and in-flight data under 10% loss. The in-flight data closely tracks the window, showing that the sender remains backlogged. Shaded intervals denote the DEC phase.

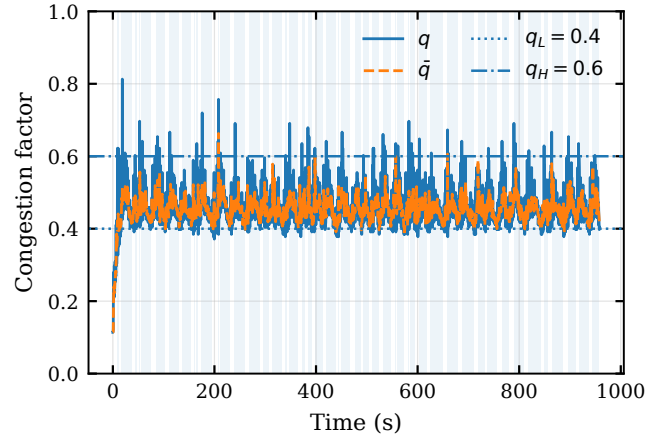


Figure 3: Instantaneous congestion likelihood q , its EMA $\bar{q}$, and the configured hysteresis thresholds. Short excursions of q trigger phase changes while $\bar{q}$ represents persistent pressure. Shading denotes the DEC phase.

system also contains random loss, ACK timing, retransmission, and EMA state.

5.4 Loss-Cost Ablation

NoLossCost completed the file in 727.451 s at 1.153 Mbit/s, but its instrumented behavior was qualitatively different. Its window grew to 496,670 bytes, q remained near 0.132, and the controller stayed in INC. It reported 154,890,042 protocol bytes to deliver 104,857,600 application bytes, corresponding to 67.70% payload efficiency and 47.7% excess traffic relative to the delivered payload. The higher goodput was therefore obtained by aggressive over-sending, not by the intended loss-responsive regulation. Removing $\hat{p}_l T_R$ cuts the feedback path from recovery pressure to the likelihood; on a path with little raw RTT inflation, the nonlinear controller then has

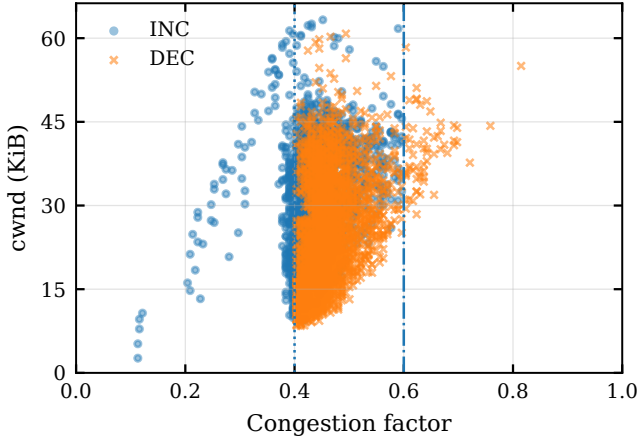


Figure 4: Window-likelihood state space for the same run. INC and DEC samples overlap within $0.4 < q < 0.6$, demonstrating phase memory: the instantaneous likelihood alone does not determine the active branch.

Table 2: Two-flow 300-second sharing over the common 5 Mbit/s bottleneck.

Loss	Pair	Flow 1 Goodput (Mbit/s)	Flow 2 Goodput (Mbit/s)	Total Goodput (Mbit/s)	Jain
10%	PROB-PROB	0.9189	0.9510	1.8699	0.9997
10%	PROB-AIMD	0.9558	0.4330	1.3888	0.8759
0%	PROB-AIMD	0.0090	4.7162	4.7252	0.5019

no effective mechanism for entering DEC. We therefore treat the ablation as evidence that the loss-cost term is structurally necessary, not as a viable replacement for SCP-PROB.

5.5 Fairness and Coexistence

Two homogeneous SCP-PROB flows obtained 49.14% and 50.86% of delivered goodput, producing $J = 0.9997$. This supports the same-signal fairness tendency established in Section 3.3 for the evaluated symmetric configuration. Under 10% loss, SCP-PROB obtained 68.8% of delivered bytes when competing with SCP-AIMD, whose loss-triggered response is repeatedly activated by random loss.

The coexistence result reversed on the loss-free path. With the deep 1000-packet queue, AIMD obtained approximately 99.8% of delivered bytes; two runs produced Jain indices 0.50194 and 0.50189. A diagnostic trace observed SRTT as high as 1.77 s. The result is consistent with a loss-based flow maintaining a deep queue while the likelihood-driven flow responds earlier to RTT inflation and yields capacity. Strong intra-protocol fairness does not imply fair coexistence with a controller driven by a different signal.

6 Discussion and Limitations

6.1 What the Experiments Establish

The experiments support three claims. First, SCP-PROB is a functioning new congestion-control algorithm rather than a transport-specific benchmark trick: replacing only SCP’s controller changes the high-loss result by more than a factor of two. Second, the completion-time likelihood allows the controller to remain responsive to loss without directly halving its window on every loss event. Third, the window-dependent update provides strong fairness between the two homogeneous flows evaluated here.

The NoLossCost result illustrates why goodput alone is insufficient. A more aggressive sender can improve application delivery while spending more protocol traffic and abandoning the intended loss-responsive cycle. In the evaluated ablation, the recovery-cost term was the component that moved q into the controller’s switching region.

6.2 Limitations

The present evaluation is intentionally preliminary.

- Each main configuration was run once, except the loss-free mixed-flow case, which was repeated twice. The results therefore lack confidence intervals across random seeds.
- The evaluation uses one emulated bottleneck rate, one propagation RTT, one deep queue, and at most two competing flows. It does not establish RTT fairness, many-flow behavior, or Internet-wide deployability.
- The recovery proxy T_R uses the minimum retransmission timeout rather than the exact recovery delay of each loss. The Q16 logistic parameters are engineering-calibrated and q is not a calibrated posterior probability.
- Receiver scheduling, application processing, ACK-path loss, and retransmission ambiguity can enter the observed RTT and loss signals.
- The analysis covers frozen-signal branch properties. It does not prove global stability, existence of a limit cycle, or global fairness for the complete networked system.
- The loss-free coexistence experiment exposes starvation against AIMD on a deeply buffered path. A future controller needs a robust mechanism for distinguishing self-induced pressure from a queue maintained by a more aggressive competing flow.
- Experiments were conducted with Linux namespaces and netem, not a large-scale real-Internet deployment.

7 Related Work

Classic TCP congestion avoidance uses loss as the primary congestion signal and applies additive increase and multiplicative decrease [1, 2]. CUBIC replaces Reno’s linear time evolution with a cubic window function for high-bandwidth and long-delay paths [3]. These algorithms remain effective on conventional queues, but random non-congestive loss can invoke the same response as queue overflow.

Vegas and FAST use RTT inflation to infer queue buildup before packet loss [4, 5]. Their lower-delay behavior can become a competitive disadvantage when sharing a deep queue with a loss-based

flow. Copa combines delay-based control with an explicit mode for coexistence with buffer-filling traffic [6]. The starvation observed in our loss-free PROB-AIMD experiment is therefore a known class of cross-signal coexistence problem, although SCP-PROB differs by incorporating recovery cost into its primary signal.

TCP Westwood estimates available bandwidth after loss, while TCP Veno uses queue information to distinguish likely congestion loss from random wireless loss [7, 8]. SCP-PROB does not classify each loss as one type or the other. Instead, it prices observed recovery pressure in time units and combines it continuously with RTT residual.

BBR estimates bottleneck bandwidth and propagation RTT and controls sending from a path model rather than using loss as its primary signal [9]. SCP-PROB is simpler and window-based: it does not estimate bottleneck bandwidth, and its bounded likelihood directly drives a nonlinear hysteretic update.

8 Conclusion

This paper introduced SCP-PROB, a new congestion-control algorithm that projects RTT inflation and loss-recovery pressure into a unified completion-time residual, maps the residual to a bounded likelihood, and uses that likelihood in a hysteretic nonlinear window controller. In a user-space reliable transport, the algorithm preserved near-line-rate clean-path performance and substantially outperformed SCP-AIMD and TCP CUBIC under severe independent bidirectional loss. It also achieved near-perfect

two-flow intra-protocol fairness. The results simultaneously expose two open problems: calibration currently trades utilization for control efficiency, and coexistence with loss-based flows on deep loss-free queues can be poor. Future work will address adaptive calibration, heterogeneous RTTs, shallower queues, many-flow experiments, and real-network deployment.

References

- [1] Van Jacobson. Congestion avoidance and control. In *Proceedings of ACM SIGCOMM*, pages 314–329, 1988.
- [2] Mark Allman, Vern Paxson, and Ethan Blanton. TCP congestion control. RFC 5681, RFC Editor, 2009.
- [3] Sangtae Ha, Injong Rhee, and Lisong Xu. CUBIC: A new TCP-friendly high-speed TCP variant. *ACM SIGOPS Operating Systems Review*, 42(5):64–74, 2008.
- [4] Lawrence S. Brakmo and Larry L. Peterson. TCP Vegas: End to end congestion avoidance on a global Internet. *IEEE Journal on Selected Areas in Communications*, 13(8):1465–1480, 1995.
- [5] David X. Wei, Cheng Jin, Steven H. Low, and Sanjay Hegde. FAST TCP: Motivation, architecture, algorithms, performance. *IEEE/ACM Transactions on Networking*, 14(6):1246–1259, 2006.
- [6] Venkat Arun and Hari Balakrishnan. Copa: Practical delay-based congestion control for the Internet. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 329–342, 2018.
- [7] Saverio Mascolo, Claudio Casetti, Mario Gerla, Medy Y. Sanadidi, and Ren Wang. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links. In *Proceedings of ACM MobiCom*, pages 287–297, 2001.
- [8] Cheng Peng Fu and Soung Chang Liew. TCP Veno: TCP enhancement for transmission over wireless access networks. *IEEE Journal on Selected Areas in Communications*, 21(2):216–228, 2003.
- [9] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. BBR: Congestion-based congestion control. *ACM Queue*, 14(5):20–53, 2016.
- [10] Raj Jain, Dah-Ming Chiu, and William R. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report DEC-TR-301, Digital Equipment Corporation, 1984.